

统一 A / A+ 的 Semantic Obligation 颗粒度： 从“LLM 自由拆约束”转向可规范化的语义谓词体系

核心结论

针对你现在遇到的 A 与 A+ 问题，我认为我们已经可以把症结说得非常明确：

现在真正出问题的不是 LLM “漏标了几个 constraint”，而是我们还没有规定 semantic obligation 的 canonical unit 到底是什么。

A 的解析把 “lost sales” 语义展开成了 `shortage_only_when_stock_exhausted` 和 `unmet_consumption_is_not_recoverable`；A+ 却把同一类语义展开成了 `shortage_cannot_exceed_daily_consumption`。与此同时，A 把 initial-day balance 和 interday balance 分开，A+ 也分开，但 lead time 又被直接揉进 balance；A+ 的 day-5 procurement upper bound 则被合并进了 daily maximum family。换言之，当前 LLM 同时在做语义理解、逻辑推论和 **formulation decomposition**，而我们没有规定三者的边界。 [filecite\turn0file0](#) [filecite\turn0file1](#)

如果直接拿这样的输出计算 mean overlap、spectral coupling、treewidth、hubness 等指标，那么指标会不可避免地受到 **annotation decomposition choice** 污染：同一个 operational semantics，只因为模型这次多拆一个 hyperedge、下次少拆一个 hyperedge，complexity 就改变了。

这不是小修 prompt 能解决的问题。我们需要先定义一个**规范化语义中间层**。

我的建议是把之前的 `semantic obligation` 重新正式定义为：

Semantic Obligation = canonical relational predicate family over canonical semantic entities

它应当对以下变化保持不变：

paraphrase, numeric values, units, index renaming, algebraic decomposition, solver encoding.

但是它必须保留真正改变 operational meaning 的东西：

scope, temporal lag, conditionality, exception structure, entity interaction.

最重要的设计改变是：

Obligation Family $\rightarrow$ Obligation Clauses $\rightarrow$ Algebraic Realization

我们以后**绝对不能**再把这三个层次混成一个 `obligations` 数组。

这个思路与 Constraint Programming 中“一个高层 global constraint / predicate 可以有許多不同 decomposition”的经典区分非常契合。MiniZinc 明确允许一个 predicate 表示一个复杂高层 constraint，而 flattening 时它可以被展开成许多低层约束；Global Constraint Catalog 也明确把 constraint 作为具有独立 semantics 的对象，并记录其逻辑、图和自动机等不同 reformulations。^① CSP 文献进一步说明，constraint-variable incidence hypergraph 的结构本身可以决定或强烈影响问题的结构复杂性，因此，如果我

们准备研究 hypergraph complexity, 首先必须保证 hyperedge 本身具有稳定语义, 而不是某次 **formulation** 的偶然拆分结果。 2

这也是 NL4Opt 当初采用 meaning representation 而不是直接从文本跳到 solver code 的基本思想: 先识别语义实体, 再生成 canonical meaning representation; 而 2026 年的 IR2Solve 又进一步证明 schema-constrained intermediate representation 和 deterministic verification 对 autoformulation 很有价值。 3

所以, 对我们而言, 现在最正确的方向不是让更强 LLM “更仔细地列 constraint”, 而是冻结一个 **annotation algebra**, 让强 LLM 只能往这个 algebra 里面填。

为什么 A 与 A+ 现在的标注必然不一致

先把两个文件严格对齐。

A 的 LLM 输出包含三个 semantic entities:

$\{\text{shipment}, \text{ending_stock}, \text{unmet_consumption}\},$

并产生七个 obligations, 其中包括 minimum shipment、maximum shipment、first-day balance、interday balance、storage capacity、`shortage_only_when_stock_exhausted` 和 `unmet_consumption_is_not_recoverable`。filecite turn0file0

A+ 仍然本质上是同一 inventory / lost-sales problem family, 却将实体命名成:

$\{\text{procurement}, \text{stock}, \text{shortage}\},$

七个 obligations 则变成 initial balance、lagged balance、minimum procurement、maximum procurement、storage capacity、final-day minimum stock 和 `shortage_cannot_exceed_daily_consumption`。filecite turn0file1

这里至少存在六种不同性质的 annotation instability。

同一实体被 lexical wording 控制了

A 用 `shipment`, A+ 用 `procurement`。

但从 operational semantics 看, 两者都是:

$Q_t = \text{amount ordered/requested at time } t.$

因此 node ID 绝不能来自题目 noun phrase。否则:

“order / procurement / shipment request / replenishment quantity / purchase amount”

可能变成五个不同 semantic nodes。

这违背了我们使用 canonical entities 的初衷。

更进一步, 我建议把 “canonical semantic **decision** entities” 这个名称稍微扩宽成:

Canonical Semantic Endogenous Entities

因为 `stock` 和 `shortage` 严格说不是 control decision；它们分别是 state 和 outcome/slack。若所有 node 都称为 decision entity，后面遇到 routing arrival time、scheduling completion time、inventory receipt、network flow state 等会越来越别扭。

建议固定：

`entity_role` $\in$ {decision, state, derived_flow, service_outcome, auxiliary-semantic}.

lead time 被错误地揉进 inventory balance

A 中 “instant delivery” 被直接揉进：

$$I_t - U_t = I_{t-1} + Q_t - D_t.$$

A+ 中 “一天 lead time” 又被揉成：

$$I_t - U_t = I_{t-1} + Q_{t-1} - D_t.$$

🔗filecite🔗turn0file0🔗 🔗filecite🔗turn0file1🔗

数学上这当然可以。

语义规范化上却不好。

因为这样会把两个本来不同的 semantic mechanisms 混在一起：

$$\underbrace{\text{order} \rightarrow \text{receipt}}_{\text{delivery timing}}$$

和

$$\underbrace{\text{previous stock} + \text{receipt} - \text{demand} \rightarrow \text{new stock/shortage.}}_{\text{material conservation}}$$

只要拆开，我们立即得到稳定 ontology：

$$Q_t = \text{order placed},$$

$$R_t = \text{physical receipt available},$$

$$I_t = \text{end-of-period inventory},$$

$$U_t = \text{unmet demand}.$$

然后 A 与 A+ 都使用同一个 material-balance obligation：

$$I_t - U_t = I_{t-1} + R_t - D_t$$

真正不同的只是 receipt-timing obligation：

A:

$$R_t = Q_t$$

A+:

$$R_{t+1} = Q_t.$$

这是非常重要的 canonicalization。

也就是说，“lead-time complexity”从 balance formula 的偶然 algebraic difference，变成一个独立、可解释、可计算的 temporal semantic relation。

同一句 lost-sales semantics 被 LLM 推出了不同的逻辑后果

这是现在最危险的问题。

A 根据:

unsatisfied consumption is lost and cannot be recovered later

生成了:

$$U_t > 0 \Rightarrow I_t = 0$$

即 shortage_only_when_stock_exhausted，同时生成“shortage 不向未来 recovery”的 obligation。
filecite[turn0file0]

A+ 面对几乎同样的:

unsatisfied consumption is lost and cannot be deferred to a later day

却没有生成上述两个，而产生:

$$U_t \leq D_t$$

即 shortage_cannot_exceed_daily_consumption。filecite[turn0file1]

这暴露出一个根本问题:

我们现在在让 LLM 对题面做 logical closure。

而 logical closure 是绝对不能用来定义 hyperedges 的。

因为一句:

$$0 \leq x \leq 10$$

可以推出:

$$x \leq 11, \quad x \leq 12, \quad 2x \leq 20, \quad x + 0 \leq 10, \dots$$

如果“所有正确逻辑推论”都有资格成为 semantic obligation，就不存在唯一 graph。

我们必须规定：

semantic graph stores a generating semantic basis, not its logical closure

这和 global-constraint 的思想非常接近：高层 predicate 被视为一个语义关系，而不是把它能够推出的所有低层条件分别列成新的 constraints。 ⁴

因此，在 A/A+ 中：

shortage_cannot_exceed_daily_consumption

不应该作为一个独立 primitive semantic obligation。

它应该被记录为：

derived_property

因为 `unmet_demand` 被定义成 “the unmet portion of today's demand”，那么

$$0 \leq U_t \leq D_t$$

是它的 definitional consequence。

同样，

$$U_t > 0 \Rightarrow I_t = 0$$

也不应该由 LLM 自由生成 primitive hyperedge。它比 $U_t \leq D_t$ 更强，依赖一个额外语义假设：

available inventory must always be used to satisfy current demand before shortage is declared.

这在经典 lost-sales inventory interpretation 下通常合理，但题面并没有明确说 “rationing is prohibited” 或 “all available stock must be used before any demand is lost”。因此第一版 annotation 最安全的做法是：

不进入 primitive SOH

并记录：

```
{
  "status": "domain_convention_or_derived",
  "count_in_semantic_hypergraph": false
}
```

这一步会直接消灭 A 的 o6 与 A+ 的 o7 这种随机不一致。

first-day balance 与 interday balance 是 boundary artifact

A:

- first_day_stock_balance
- interday_stock_balance

A+:

- initial_day_inventory_balance
- lagged_daily_inventory_balance

filecite[turn0file0] filecite[turn0file1]

从 implementation 角度拆开很自然。

从 semantic-obligation 角度，不应该。

我们只需要定义：

$$I_0 = \text{initial inventory parameter}$$

然后统一：

$$I_t - U_t = I_{t-1} + R_t - D_t, \quad t = 1, \dots, T.$$

这样 $t = 1$ 自动引用 parameter I_0 。

所以：

$$\text{first-day balance} \neq \text{new semantic obligation.}$$

它只是：

$$\text{boundary realization of the same conservation law.}$$

这与结构化建模语言将高层 predicate 和其 scalar / flattened realization 区分开的思想一致；IR2Solve 甚至有意采用 scalarized constraints 来降低 code/quantification error，但那是 solver-facing IR 的设计，而不是我们应采用的 semantic granularity。 ⁵

day-5 upper bound 应该是同一 family 下的 override，而不是全新 taxonomy

A+:

$$Q_t \leq 76, \quad t = 1, \dots, 4,$$

$$Q_5 \leq 39.$$

当前 A+ LLM 把它们合成一个 maximum_dai ly_procurement obligation。 filecite[turn0file1]

这个方向其实基本正确。

但单纯合成又会损失一个重要信息：

A+ 多了一个 special-case business rule。

所以我们需要 family/clause 两级。

Obligation family:

```
ORDER_UPPER_BOUND
```

下面有 clauses。

A:

```
scope = days 1..5  
upper_bound = 35
```

A+:

```
clause 1:  
scope = days 1..4  
upper_bound = 76  
  
clause 2:  
scope = day 5  
upper_bound = 39  
modifier = override
```

于是：

$$N_{\text{family}}$$

不被无意义地增加，

但：

$$N_{\text{clauses}}, \quad N_{\text{exceptions}}$$

增加。

这恰恰可能是以后 operational complexity 中一个非常有价值的维度：

scope / exception heterogeneity

而不是把 “day 5 max” 粗暴视作又一条普通 constraint。

A+ 并不是严格意义的 “A + 几个 constraints”

这一点对后面的 experiment 很重要。

A 是：

$$L = 0$$

instant delivery。

A+ 是：

$$L = 1.$$

这不是简单向 feasible region 追加一个约束，而是修改了原来的 **operational transition law**。A+ 还改变了容量、初始库存、成本、需求数值和 procurement bounds，并增加 terminal stock floor。

filecite[turn0file0] filecite[turn0file1]

因此更准确地说：

$$A+ = \text{mutate}(A, \text{lead-time}) + \text{augment}(A, \text{exceptions} / \text{terminal rule}) + \text{reparameterize}(A).$$

OPT-Engine 本身就是为了研究 controllable structural complexity 而设计的，并明确把 linguistic variance、objective perturbation 与 constraint disturbances 分开；其公开说明也强调 constraint formulation 是当前 solver-integrated LLM 的主要 bottleneck。 ⁶

所以未来我们测试 “augmentation invariance” 时，必须进一步区分：

pure additive augmentation

和

structural mutation.

否则我们会错误地期待：

$$SOH(A) \subseteq SOH(A+),$$

而这个例子其实并不满足严格的 additive premise。

一个可以冻结下来的 canonical 定义

我建议从现在开始，把 Semantic Obligation Hypergraph 的定义改成下面这个版本。

Canonical Semantic Entity

定义：

Canonical Semantic Entity 是在 operational problem 中具有独立 operational identity 的 endogenous quantity/state/decision/outcome family；其 identity 由其 operational

role、index semantics 和 value semantics 决定，而不是由题面使用的名词、数学变量名或 solver implementation 决定。

于是：

shipment request $\sim$ procurement request $\sim$ order quantity

只要它们都是：

ORDER_QUANTITY[time].

建议 entity canonicalization signature 为：

$\sigma_E(e) = (\text{semantic role, business object, index-role tuple, temporal phase, endogenous type})$

而 numerical units 不进入 identity。

因此 gallon / liter 不改变 node。

Semantic Obligation

我建议正式写成：

A canonical semantic obligation is a parameterized relational predicate family over canonical semantic entities that expresses one primitive operational requirement, transition law, admissibility rule, or policy required to characterize admissible operational trajectories, after abstracting away lexical realization, numerical parameter values, index instantiation, algebraic reformulation, and solver-specific encoding.

中文：

一个 canonical semantic obligation，是定义在 canonical semantic entities 上的参数化关系谓词族，用于表达一个 primitive 的 operational requirement、transition law、admissibility rule 或 policy；其 identity 不受语言表达、具体数值、逐索引展开方式、代数等价变换或 solver 实现方式影响。

可以形式化为：

$$o = (P, \mathcal{R}, \mathcal{Q}, \mathcal{S}, \mathcal{T}, \mathcal{C}, \Theta)$$

其中：

P = canonical predicate,

$\mathcal{R}$ = entity role signature,

$\mathcal{Q}$ = quantification/index pattern,

$\mathcal{S}$ = scope,

$\mathcal{T}$ = temporal/index transformation,

$\mathcal{C}$ = condition / qualifier structure,

Θ = parameters.

但是 canonical identity:

$$\sigma_O(o) = (P, \mathcal{R}, \mathcal{Q}, \mathcal{T}, \text{qualifier class})$$

不包含实际 numerical values。

例如:

$$Q_t \leq 35$$

和

$$Q_t \leq 76$$

具有同一个 canonical identity:

ORDER_UPPER_BOUND.

Obligation Family 与 Obligation Clause 必须分离

这是解决 A/A+ 的关键。

一个:

Obligation Family

表示 canonical predicate。

一个:

Obligation Clause

表示这个 predicate 在某个 index region / condition 下的具体适用规则。

因此:

```
{
  "family": "ORDER_UPPER_BOUND",
  "clauses": [
    {
      "scope": "t = 1..4",
      "bound": 76
    },
    {
      "scope": "t = 5",
      "bound": 39,
      "relation_to_parent": "override"
    }
  ]
}
```

```
}  
]  
}
```

而不是：

```
maximum procurement  
final-day maximum procurement
```

由 LLM 自己决定拆不拆。

原始语义和推论必须分层

建议强制每个 candidate statement 进入下面五类之一：

`primitive_explicit`

题面直接规定的 operational rule。

`primitive_definitional`

为了让题面中一个明确 operational concept 有确定 semantics 而不可避免的定义性关系。

`derived_consequence`

由 primitives 可以推出，但不应成为新 hyperedge。

`domain_convention`

依赖库存、调度、routing 等领域惯例，而题面没有明确指定。

`formulation_choice`

big-M、linearization、auxiliary variable、symmetry breaking、redundant valid inequality 等。

SOH 主图只允许：

`primitive_explicit ∪ primitive_definitional`

进入。

这对于避免 annotation explosion 至关重要。Constraint Programming 文献之所以区分 high-level global constraints 与它们的 decomposition，也是因为同一个语义 relation 可以存在多种等价或近等价的 modeling realization。 ⁷

A 与 A+ 应该怎样被统一规范化

在这个具体 inventory family 上，我建议第一版 ontology 强制使用四个 canonical entities。

Canonical entity	A 原文本概念	A+ 原文本概念	role
Q_t <code>order_quantity</code>	shipment requested	procurement request	decision flow
R_t <code>receipt_quantity</code>	shipment delivered instantly	procurement becoming available after lead time	derived flow
I_t <code>end_inventory</code>	ending stock	stock carried/available	state
U_t <code>unmet_demand</code>	deficit/unmet consumption	shortage/unmet consumption	service outcome

A/A+ 原始 LLM 都没有独立引入 R_t ，而是把 receipt 和 procurement 混在了一起。filecite[turn0file0]filecite[turn0file1]

我认为我们**应该主动规范化出 R_t** 。

这不是 formulation auxiliary variable，而是现实中的一个独立 operational event：

$$\text{order placement} / \text{physical receipt} =$$

当 lead time 不为零时，两者显然不同。

而在 A 中只是：

$$R_t = Q_t.$$

这样同一个 ontology 可以覆盖：

$$L = 0, 1, 2, \dots$$

而不用每次重新发明 obligation taxonomy。

规范化后的 A

建议 canonical semantic obligations 为：

Family	Canonical semantics	support
<code>ORDER_LOWER_BOUND</code>	每期 order 至少为允许下界	$\{Q\}$
<code>ORDER_UPPER_BOUND</code>	每期 order 不超过允许上界	$\{Q\}$
<code>ORDER_RECEIPT_TIMING</code>	order 按规定 lead time 转化为 available receipt	$\{Q, R\}$
<code>INVENTORY_FLOW_CONSERVATION</code>	previous inventory + receipts 与 demand、ending inventory、unmet demand 守恒	$\{R, I, U\}$
<code>STORAGE_CAPACITY</code>	physically held inventory 不超过容量	$\{I\}$
<code>LOST_SALES_DISPOSITION</code>	当期 unmet demand 丢失，不成为未来 backlog	$\{U\}$

其参数化 semantics 是：

$$\begin{aligned}
Q_t &\geq 6, \\
Q_t &\leq 35, \\
R_t &= Q_t, \\
I_t - U_t &= I_{t-1} + R_t - D_t, \quad I_0 = 42, \\
I_t &\leq 55,
\end{aligned}$$

以及：

U_t represents same-period unmet demand and is not carried into $t + 1$.

A 的文本明确给出了 instant availability、lost unrecoverable demand、end-of-day capacity 等语义。
turn0file0

注意：

$$U_t \leq D_t$$

不单独新增 family。

同样：

$$U_t > 0 \Rightarrow I_t = 0$$

暂时也不新增 family。

它们进入：

derived_properties / convention_candidates

而不是 primitive SOH。

规范化后的 A+

A+ 使用**完全相同**的前六个 family。

变化仅仅发生在 clauses / parameters / index transforms。

`ORDER_LOWER_BOUND`：

$$Q_t \geq 4.$$

`ORDER_UPPER_BOUND`：

$$Q_t \leq \begin{cases} 76, & t = 1, \dots, 4, \\ 39, & t = 5. \end{cases}$$

所以：

$$N_{\text{family}} = 1$$

但：

$$N_{\text{upper-bound clauses}} = 2.$$

ORDER_RECEIPT_TIMING :

$$R_{t+1} = Q_t.$$

也就是：

$$\text{lag} = 1$$

而不是 A 的：

$$\text{lag} = 0.$$

INVENTORY_FLOW_CONSERVATION 仍然完全一样：

$$I_t - U_t = I_{t-1} + R_t - D_t.$$

STORAGE_CAPACITY 仍然是同一个 canonical family，但 A+ 说的是：

physically held stock **at any moment** ≤ 48

而 A 只说 end-of-day stock ≤ 55 。      

因此 canonical predicate 不变：

STORAGE_CAPACITY

但 scope 必须不同：

A:

"measurement_scope": "end_of_period"

A+:

"measurement_scope": "all_physical_stock_measurement_phases"

并且 A+ 这里继续保留：

"scope_resolution": "partially_unresolved"

因为如果 receipt 在消费前到达，那么 “at any moment” 可能要求检查：

$$I_{t-1} + R_t \leq 48,$$

而不仅仅是：

$$I_t \leq 48.$$

A+ 原解析自己其实也注意到了这个 ambiguity。 [filecite turn0file1](#)

`LOST_SALES_DISPOSITION`：

两题都应该有：

$$U_t \text{ is lost, not backlogged.}$$

这正是当前 A+ parser 漏掉而应规范化恢复的部分。 [filecite turn0file1](#)

最后 A+ 多一个 genuine new obligation：

`TERMINAL_INVENTORY_FLOOR`

即：

$$I_5 \geq 15$$

——但 day-5 “physically available stock” 是 consumption 前还是后的 state 仍有语义 ambiguity，因此应保留 phase qualifier，而不是让 LLM 擅自确定。A+ 原解析也对此进行了 unresolved 标记。

[filecite turn0file1](#)

于是最终得到非常干净的结构：

$$\mathcal{F}_A = \{LB_Q, UB_Q, TIMING, BALANCE, CAPACITY, LOST_SALES\}$$

而：

$$\mathcal{F}_{A+} = \mathcal{F}_A \cup \{TERMINAL_FLOOR\}$$

但其中：

$$TIMING_A : lag = 0,$$

$$TIMING_{A+} : lag = 1,$$

并且：

$$UB_{A+}$$

具有 special-day override。

这就是我认为我们真正想要的“统一颗粒度”。

这对我们的 complexity metric 有一个非常重要的新启示

规范化完成后，会出现一个很有意思的结果：

A+ 明显比 A 更难理解，但它不一定具有明显更高的纯 support-level constraint coupling。

因为忽略 unary edges 后，两题最核心的 coupling structure 都大致是：

$$Q \leftrightarrow R$$

以及：

$$\{R, I, U\}.$$

换言之，family-level primal topology 可能几乎相同。

这不是 metric 失败。

恰恰相反，它告诉我们：

$$\boxed{\text{constraint complexity} /} = \text{coupling alone.}$$

A+ 的新增难度主要落在至少三个其他维度上。

第一是 **temporal displacement**：

$$L : 0 \rightarrow 1.$$

第二是 **scope heterogeneity / exception structure**：

$$Q_t \leq 76$$

变成：

$$Q_t \leq 76 \ (t < 5), \quad Q_5 \leq 39.$$

第三是 **additional terminal obligation**：

$$I_5 \geq 15.$$

所以我们未来的 semantic complexity vector 很可能需要从：

$$\mathbf{C}_{\text{coupling}}$$

扩展为类似：

$$\boxed{\mathbf{C}_{\text{semantic}} = (C_{\text{interaction}}, C_{\text{temporal}}, C_{\text{scope}}, C_{\text{exception}}, C_{\text{rule-load}}, C_{\text{logical}})}$$

其中 coupling graph metrics——overlap、spectral、treewidth、hubness、cycles——主要服务于：

$$C_{\text{interaction}}.$$

而不是强迫它们解释所有 semantic difficulty。

CSP 研究给了我们很强的理论理由研究 constraint interaction structure: constraint-incidence hypergraph、treewidth、hypertree-width 等结构量确实与求解结构复杂度存在深刻关系；但它们从来没有声称“所有 constraint difficulty 都等于 graph density”。⁸

这也与 OPT-Engine 的结果形成了一个非常有价值的联系：OPT-Engine 观察到 constraint formulation 是主要 bottleneck，但其 complexity scaling 主要还是依靠 variable/constraint counts、问题规模以及 controlled perturbations；它并没有给出我们现在讨论的这种 canonical semantic-interaction definition。⁶

这实际上让我们的研究空间更清晰。

当前两个原始解析已经会制造明显的假 metric 差异

甚至不用 sophisticated metric 就能看出来。

如果把当前两个 LLM 输出里的三个实体分别 lexical-align：

$$\begin{aligned} \textit{shipment}/\textit{procurement} &\rightarrow Q, \\ \textit{ending_stock}/\textit{stock} &\rightarrow I, \\ \textit{unmet}/\textit{shortage} &\rightarrow U, \end{aligned}$$

然后错误地直接把现有七个 obligations 全部作为 hyperedges，那么 A 的 edge-support multiset 是：

$$\{Q\}, \{Q\}, \{Q, I, U\}, \{Q, I, U\}, \{I\}, \{I, U\}, \{U\},$$

而 A+ 是：

$$\{I, U\}, \{Q, I, U\}, \{Q\}, \{Q\}, \{I\}, \{I\}, \{U\}.$$

这些都直接来自当前两个解析。filecite[turn0file0] filecite[turn0file1]

用我们之前定义的 constraint-intersection density：

$$D_{\cap} = \frac{|\{(e_i, e_j) : e_i \cap e_j \neq \emptyset\}|}{\binom{m}{2}} = |\emptyset|$$

计算，会得到：

$$D_{\cap}(A) = \frac{14}{21} \approx 0.667,$$

而：

$$D_{\cap}(A+) = \frac{11}{21} \approx 0.524.$$

也就是说：

A + 明明增加了 *leadtime* 和 *terminalrule*，未经 *canonicalization* 的 *metric* 反而会告诉我们“*constraintoverlap*”

这几乎完全是 annotation decomposition artifact。

这个小例子非常重要。

它实际上证明了我们正式大规模跑 metric 之前，**canonicalization 不是 optional preprocessing，而是 metric validity 的前提。**

建议冻结成 SOH-Norm 的数据结构与强模型标注规则

我建议下一版 schema 不再直接使用现在这种：

```
"obligations": [  
  {  
    "canonical_name": "...",  
    "obligation_type": "..."  
  }  
]
```

因为 `obligation_type` 目前让 LLM 自由选择 `capacity / resource_limit / service_requirement / policy / logical_implication`，这些 label 并没有规定 canonical identity。

应改成四层。

Entity registry

例如：

```
{  
  "entities": [  
    {  
      "entity_id": "order_quantity",  
      "entity_role": "decision_flow",  
      "business_object": "additive",  
      "indices": ["time"],  
      "temporal_phase": "order_placement",  
      "aliases_in_text": [  
        "shipment volume",  
        "procurement volume"  
      ]  
    },  
    {  
      "entity_id": "receipt_quantity",  
      "entity_role": "derived_flow",  
      "business_object": "additive",  
      "indices": ["time"],  
      "temporal_phase": "physical_availability"  
    },  
    {  
      "entity_id": "order_quantity",  
      "entity_role": "decision_flow",  
      "business_object": "additive",  
      "indices": ["time"],  
      "temporal_phase": "order_placement",  
      "aliases_in_text": [  
        "shipment volume",  
        "procurement volume"  
      ]  
    },  
    {  
      "entity_id": "receipt_quantity",  
      "entity_role": "derived_flow",  
      "business_object": "additive",  
      "indices": ["time"],  
      "temporal_phase": "physical_availability"  
    }  
  ]  
}
```

```

    "entity_id": "inventory",
    "entity_role": "state",
    "indices": ["time"],
    "temporal_phase": "end_of_period"
  },
  {
    "entity_id": "unmet_demand",
    "entity_role": "service_outcome",
    "indices": ["time"]
  }
]
}

```

Obligation family

```

{
  "family_id": "order_receipt_timing",
  "structural_class": "temporal_link",
  "predicate": "RECEIPT_FROM_ORDER",
  "argument_roles": [
    "order_quantity",
    "receipt_quantity"
  ]
}

```

注意：

`structural_class`

只是 coarse taxonomy。

真正决定 canonical identity 的是：

predicate + argument roles + index-transform signature

不能再由 `capacity` 这种 broad type 决定。

Obligation clauses

A:

```

{
  "family_id": "order_receipt_timing",
  "clauses": [
    {
      "scope": "t in days",
      "index_relation": {

```

```

    "receipt_time": "t",
    "order_time": "t"
  },
  "lag": 0,
  "source_status": "primitive_explicit"
}
]
}

```

A+:

```

{
  "family_id": "order_receipt_timing",
  "clauses": [
    {
      "scope": "order_time t such that t+1 is in horizon",
      "index_relation": {
        "receipt_time": "t+1",
        "order_time": "t"
      },
      "lag": 1,
      "source_status": "primitive_explicit"
    }
  ]
}

```

Upper bound A+:

```

{
  "family_id": "order_upper_bound",
  "clauses": [
    {
      "scope": "t in {1,2,3,4}",
      "upper_bound": 76,
      "modifier": "base_rule"
    },
    {
      "scope": "t = 5",
      "upper_bound": 39,
      "modifier": "override"
    }
  ]
}

```

这使 exception complexity 成为机器可读结构。

Derived / unresolved layer

比如：

```
{
  "derived_semantics": [
    {
      "statement": "unmet_demand[t] <= demand[t]",
      "derivation_basis": "unmet_demand denotes the unmet portion of current demand",
      "status": "definitional_consequence",
      "count_as_obligation": false
    },
    {
      "statement": "unmet_demand[t] > 0 implies inventory[t] = 0",
      "status": "domain_convention_candidate",
      "count_as_obligation": false
    }
  ]
}
```

以及：

```
{
  "unresolved_items": [
    {
      "issue": "terminal inventory measurement phase",
      "candidate_interpretations": [
        "before day-5 consumption",
        "after day-5 consumption"
      ]
    }
  ]
}
```

这样我们不要求强 AI 永远正确解决 ambiguity。

我们只要求：

它不能通过偷偷选一个 interpretation 来制造 graph edge。

这是更现实的可靠性目标。

强模型 prompt 里最重要的几句话

未来 prompt 的核心不应该是：

“Extract all constraints.”

而应该类似：

Do not enumerate all mathematical constraints implied by the text. Identify only canonical primitive semantic obligation families. A semantic obligation is a high-level operational relation, not an algebraic row and not a logical consequence of another obligation.

然后明确：

Do not split a universally quantified relation into first-period and later-period obligations merely because their algebraic implementation has different boundary cases.

以及：

Represent numerical or time-specific modifications of an existing predicate as clauses or overrides under the same obligation family unless the modification introduces a new semantic relation.

以及最关键的一条：

When comparing or annotating variants of the same problem family, preserve the identity and granularity of pre-existing obligation families. An augmentation may add a clause, override a parameter/scope, modify an index relation, or introduce a genuinely new obligation family, but it must not cause unrelated existing obligations to be resegmented.

我们甚至可以把这个性质正式叫：

Augmentation Consistency

对于真正的 pure augmentation：

$$X^+ = X \oplus \Delta$$

要求：

$$C(X^+) = C(X) \oplus C(\Delta)$$

其中 C 是 canonicalizer。

更一般地，如果 augmentation 修改一个现有 rule：

$$C(X^+) = \text{Update}(C(X), \Delta_{\text{override}}) \cup C(\Delta_{\text{new}})$$

而不允许：

$$C(X^+)$$

因为多了一段文本就重新把整个 inventory logic 换一种 decomposition。

我认为这条 **Augmentation Consistency** 本身就非常有研究价值，因为 OPT-Engine 恰好提供了 controlled perturbation / constraint augmentation setting，可以天然拿来测 annotation stability。OPT-Engine 的框架就是为了 controllable complexity、canonical construction 与 perturbation validation 而设计，并报告 constraint formulation 是 SIR 的主要瓶颈。⁹

最终，我们目前可以冻结下面这个原则：

$$\boxed{\begin{array}{l} \text{Semantic topology} / \\ \text{ } / \end{array}} = \begin{array}{l} \text{solver constraint topology} \\ \text{LLM-inferred logical closure.} \end{array}$$

而应该是：

$$\boxed{\text{Semantic topology} = \text{canonical primitive operational predicate topology.}}$$

这会直接决定我们后面的 metric calculation script 应该怎样设计：**interaction coupling** 从 **canonical multi-entity obligation families** 计算；**rule count** 从 **primitive clauses** 计算；**exception complexity** 从 **overrides** 计算；**temporal complexity** 从 **index transformations/lag** 计算；**unary terminal/capacity rules** 不应该因为数量增加就伪造 **pairwise coupling**；**derived consequences** 完全不进入 graph。

我认为这一步解决以后，我们原来模糊的“constraint complexity”已经开始自然分裂成几个具有明确 operational interpretation 的轴：

$$\boxed{\text{Constraint Semantic Complexity} = \left[\begin{array}{l} \text{interaction coupling} \\ \text{temporal coupling} \\ \text{scope heterogeneity} \\ \text{exception / conditional structure} \\ \text{primitive rule load} \\ \text{logical/disjunctive structure} \end{array} \right]}$$

而 $A \rightarrow A+$ 恰好是一个非常好的 sanity check：它主要增加的是 **temporal**、**exception** 和 **terminal-rule complexity**，而不是简单的 **entity-interaction density**。这比“多三个 constraints，所以 complexity +3”更接近我们真正想定义的 operational problem solving complexity。

¹ <https://minizinc-doc-test.readthedocs.io/en/stable/predicates.html>

<https://minizinc-doc-test.readthedocs.io/en/stable/predicates.html>

² ⁸ <https://arxiv.org/abs/0911.0801>

<https://arxiv.org/abs/0911.0801>

³ <https://nl4opt.github.io/>

<https://nl4opt.github.io/>

⁴ ⁷ <https://sofdem.github.io/gccat/gccat/titlepage.html>

<https://sofdem.github.io/gccat/gccat/titlepage.html>

⁵ <https://arxiv.org/abs/2608.02641>

<https://arxiv.org/abs/2608.02641>

⁶ <https://arxiv.org/abs/2601.19924>

<https://arxiv.org/abs/2601.19924>

⁹ <https://github.com/Cardinal-Operations/OPTEngine/blob/main/README.md>
<https://github.com/Cardinal-Operations/OPTEngine/blob/main/README.md>